

debouncer/Button Debouncing/hw2test.asm

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;                                                                 ;
3  ;                      HW2TEST                                   ;
4  ;                      Homework #2 Test Code                     ;
5  ;                      EE/CS 10b                                  ;
6  ;                                                                 ;
7  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
8
9  ; Description:          This program tests the sensor functions forHomework #2.
10 ;                      It sets up the stack and calls the homework test function.
11 ;
12 ; Input:               User presses of the switches (sensors) are stored in
13 ;                      memory.
14 ; Output:              None.
15 ;
16 ; User Interface:      No real user interface. The user inputs switch presses
17 ;                      and that data along with the status function output is
18 ;                      written to memory.
19 ; Error Handling:      None.
20 ;
21 ; Algorithms:          None.
22 ; Data Structures:     None.
23 ;
24 ; Known Bugs:          None.
25 ; Limitations:         Only the last (at least) 128 switch inputs are stored.
26 ;
27 ; Revision History:
28 ;   5/17/19 Glen George          initial revision
29 ;   4/30/25 Glen George          changed function calls to match the
30 ;                               pinball machine controller board
31 ;   4/30/25 Glen George          updated comments
32 ;   4/28/26 Glen George          updated comments
33 ;   4/30/26 Aaditya Bhat        added init function
34 ;                               updated vector table
35 ;                               added includes
36
37
38
39
40 ;set the device
41 ;.device  ATMEGA64
42
43
44
45
46 ;get the definitions for the device
47 .include  "m64def.inc"
48
```

```

49 ;include all the .inc files since all .asm files are needed here (no linker)
50 .include "symbols.inc"
51
52
53
54
55
56 .cseg
57
58
59
60
61 ;setup the vector area
62
63 .org    $0000
64
65     JMP     Start           ;reset vector
66     JMP     PC              ;external interrupt 0
67     JMP     PC              ;external interrupt 1
68     JMP     PC              ;external interrupt 2
69     JMP     PC              ;external interrupt 3
70     JMP     PC              ;external interrupt 4
71     JMP     PC              ;external interrupt 5
72     JMP     PC              ;external interrupt 6
73     JMP     PC              ;external interrupt 7
74     JMP     PC              ;timer 2 compare match
75     JMP     PC              ;timer 2 overflow
76     JMP     PC              ;timer 1 capture
77     JMP     PC              ;timer 1 compare match A
78     JMP     PC              ;timer 1 compare match B
79     JMP     PC              ;timer 1 overflow
80     JMP     Timer0_ISR      ;timer 0 compare match
81     JMP     PC              ;timer 0 overflow
82     JMP     PC              ;SPI transfer complete
83     JMP     PC              ;UART 0 Rx complete
84     JMP     PC              ;UART 0 Tx empty
85     JMP     PC              ;UART 0 Tx complete
86     JMP     PC              ;ADC conversion complete
87     JMP     PC              ;EEPROM ready
88     JMP     PC              ;analog comparator
89     JMP     PC              ;timer 1 compare match C
90     JMP     PC              ;timer 3 capture
91     JMP     PC              ;timer 3 compare match A
92     JMP     PC              ;timer 3 compare match B
93     JMP     PC              ;timer 3 compare match C
94     JMP     PC              ;timer 3 overflow
95     JMP     PC              ;UART 1 Rx complete
96     JMP     PC              ;UART 1 Tx empty
97     JMP     PC              ;UART 1 Tx complete
98     JMP     PC              ;Two-wire serial interface

```

```

99      JMP      PC                      ;store program memory ready
100
101
102
103
104 ; start of the actual program
105
106 Start:                                ;start the CPU after a reset
107     LDI      R16, LOW(TopOfStack)    ;initialize the stack pointer
108     OUT      SPL, R16
109     LDI      R16, HIGH(TopOfStack)
110     OUT      SPH, R16
111
112
113     ;call any initialization functions
114     RCALL    InitVariables
115     RCALL    InitPorts
116     RCALL    InitTimer0
117     SEI
118
119
120     RCALL    SensorTest                ;do the sensor tests
121     RJMP     Start                    ;shouldn't return, but if it does, restart
122
123
124
125
126 ; SensorTest
127 ;
128 ; Description:      This procedure tests the sensor functions for Homework #2
129 ;                  (HaveSensor and GetSensor). It alternates calls to the
130 ;                  two functions, filling SensorBuf with the data it
131 ;                  receives. If the functions are working properly the
132 ;                  buffer should be filled with debounced sensor codes
133 ;                  alternating with 0xFF. The function does not test that
134 ;                  GetSensor properly blocks. The function never returns.
135 ;
136 ; Operation:        The function first initializes the buffer by filling it
137 ;                  with 0x55. It then loops calling HaveSensor and GetSensor
138 ;                  to test these functions. First GetSensor is called
139 ;                  checking HaveSensor first and the returned sensor code is
140 ;                  put in the buffer. Next HaveSensor is called. If there
141 ;                  is no sensor available (there should not be since
142 ;                  GetSensor was just called), 0xFF is written to the buffer.
143 ;                  Next GetSensor is called without waiting for a sensor to
144 ;                  be activated to check that it does wait for a new sensor
145 ;                  activation. The returned sensor code is written to the
146 ;                  buffer. Finally HaveSensor is called again and 0xFF is
147 ;                  written to the buffer if there is no sensor available.
148 ;                  Then all of this is repeated in an infinite loop.

```

```

149 ;
150 ; Arguments:      None.
151 ; Return Value:   None.
152 ;
153 ; Local Variables: R4 – index into SensorBuf
154 ; Shared Variables: None.
155 ;
156 ; Input:          None.
157 ; Output:         None.
158 ;
159 ; Error Handling:  None.
160 ;
161 ; Algorithms:      None.
162 ; Data Structures: None.
163 ;
164 ; Registers Changed: flags, R4, R16, R17, Y (YH | YL)
165 ; Stack Depth:    at least 3 bytes
166 ;
167 ; Author:         Glen George
168 ; Last Modified:  April 30, 2025
169 ;
170 ; Special Notes:  The buffer (SensorBuf) must be 256 bytes long since the
171 ;                 index is incremented and allowed to wrap at 255.
172
173 SensorTest:
174
175 STClearBuffer:                ;first clear the buffer
176     LDI    YL, LOW(SensorBuf)  ;get the start of the buffer
177     LDI    YH, HIGH(SensorBuf)
178     LDI    R17, 0x55           ;will fill buffer with 55 to start
179     CLR    R4                  ;initialize the loop counter
180
181 STClearBufferLoop:            ;loop clearing the buffer
182     ST     Y+, R17              ;initialize one byte of the buffer
183     INC    R4                  ;update the loop index
184     BRNE   STClearBufferLoop    ;and loop until fill 256 bytes
185
186     CLR    R4                  ;initialize the buffer index (again)
187
188
189 SensorTestLoop:                ;loop testing the functions
190
191 STCheckGetSensor1:            ;first call to GetSensor uses HaveSensor
192     PUSH   R4                  ;save the buffer index
193
194 STWaitLoop:                    ;loop until there is an activated sensor
195     RCALL  HaveSensor
196     BREQ   STWaitLoop          ;wait for there to be a sensor activations
197
198     RCALL  GetSensor            ;sensor is available – get it

```

```

199      POP      R4                ;get buffer index back
200      RCALL    StoreBuff         ;store sensor code in the buffer
201
202  STCheckHaveSensor1:           ;now check that HaveSensor is working
203      PUSH     R4                ;check if a sensor is still available (don't
204      RCALL    HaveSensor        ;   trash buffer index)
205      POP      R4
206
207      BRNE     STSkipFFwrite1    ;if there is no sensor available
208      LDI      R16, 0xFF          ;   write FF to buffer
209      RCALL    StoreBuff         ;store 0xFF in the buffer
210  STSkipFFwrite1:
211      ;RJMP     STCheckGetSensor2 ;do second test of GetSensor
212
213
214  STCheckGetSensor2:            ;second call to GetSensor does not use HaveSensor
215      PUSH     R4                ;save the buffer index
216      RCALL    GetSensor         ;should wait for an activated sensor
217      POP      R4                ;get buffer index back
218      RCALL    StoreBuff         ;store sensor code in the buffer
219
220  STCheckHaveSensor2:           ;now check that HaveSensor is working again
221      PUSH     R4                ;check if a sensor activation is still
222      RCALL    HaveSensor        ;   available (don't trash buffer index)
223      POP      R4
224
225      BRNE     STSkipFFwrite2    ;if there is no sensor activation
226      LDI      R16, 0xFF          ;   write FF to buffer
227      RCALL    StoreBuff         ;store 0xFF in the buffer
228  STSkipFFwrite2:
229
230
231      RJMP     SensorTestLoop    ;and keep looping forever
232
233      RET                          ;should never get here
234
235
236
237
238 ; StoreBuff
239 ;
240 ; Description:      This procedure stores the byte passed in R16 at the
241 ;                  offset in the SensorBuf buffer passed in R4. The offset
242 ;                  is updated and the new offset is returned in R4.
243 ;
244 ; Operation:       The Y register is loaded with the buffer address. The
245 ;                  passed offset is then added to this address and the
246 ;                  passed byte is stored at this location. The passed
247 ;                  offset is then incremented and returned.
248 ;

```

```

249 ; Arguments:      R4  - offset in SensorBuf at which to write the passed
250 ;                  byte.
251 ;                  R16 - byte to write to the buffer at the passed offset.
252 ; Return Value:   R4  - offset of the next location in the buffer.
253 ;
254 ; Local Variables: Y  - pointer into buffer.
255 ; Shared Variables: None.
256 ; Global Variables: None.
257 ;
258 ; Input:           None.
259 ; Output:          None.
260 ;
261 ; Error Handling:   None.
262 ;
263 ; Algorithms:       None.
264 ; Data Structures:  None.
265 ;
266 ; Registers Changed: flags, R4, R17, Y (YH | YL)
267 ; Stack Depth:     0 bytes
268 ;
269 ; Author:           Glen George
270 ; Last Modified:    April 30, 2025
271
272 StoreBuff:
273
274     LDI     YL, LOW(SensorBuf)      ;get buffer location to store sensor at
275     LDI     YH, HIGH(SensorBuf)
276
277     LDI     R17, 0                  ;for carry propagation
278     ADD     YL, R4                  ;add the passed offset
279     ADC     YH, R17
280
281     STD     Y + 0, R16              ;store the passed byte in the buffer
282
283     INC     R4                      ;update the buffer offset, wrapping at 256
284
285
286     RET                             ;all done, return
287
288
289
290
291 ;the data segment
292
293
294 .dseg
295
296
297 ; buffer in which to store sensor activations (length must be 256)
298 SensorBuf: .BYTE 256

```

```
299
300
301 ; the stack – 128 bytes
302     .BYTE    127
303 TopOfStack:    .BYTE    1                ;top of the stack
304
305
306
307
308 ; since don't have a linker, include all the .asm files
309
310 .include "init.asm"
311 .include "sensors.asm"
312
```

debouncer/Button Debouncing/symbols.inc

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;                                                                 ;
3 ;                               symbols.inc                        ;
4 ;                               Symbol & Hardware Defs           ;
5 ;                                                                 ;
6 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
7
8 ; Description:           This file defines constants and hardware-specific
9 ;                       configs for the 5x8 sensor matrix. It includes port
10 ;                       definitions, debounce timing, and state machine states.
11 ;
12 ; Input:                None.
13 ; Output:               None.
14 ;
15 ; User Interface:       None.
16 ;
17 ; Error Handling:        None.
18 ;
19 ; Algorithms:            None.
20 ; Data Structures:       None.
21 ;
22 ; Known Bugs:            None.
23 ; Limitations:           None.
24 ;
25 ; Revision History:
26 ;   4/28/26   Aaditya Bhat                added debounce constants and state machine
27 ;   5/02/26   Aaditya Bhat                added comments/documentation
28
29 ;Hardware Port Definitions: 5x8 Matrix. Rows are outputs, Columns are inputs.
30 .equ ROW_PORT = PORTG                ; Port to drive rows HIGH/LOW
31 .equ ROW_DDR  = DDRG                ; Data direction register for rows
32 .equ COL_PORT = PORTE                ; Port to enable pull-ups for columns
33 .equ COL_DDR  = DDRE                ; Data direction register for columns
34 .equ COL_PIN  = PINE                ; Port to read the actual column states
35
36 ;Debouncing Constants
37 ; The timer ISR runs every 1ms. We need 20 consecutive milliseconds of the
38 ; same switch state to consider it fully debounced.
39 .equ DEBOUNCE_TIME = 20
40
41 ;State Machine States
42 .equ STATE_IDLE      = 0            ; Waiting for a new key press
43 .equ STATE_DEBOUNCE  = 1            ; Key detected, currently verifying duration
44 .equ STATE_WAIT_RELEASE = 2        ; Key confirmed, waiting for physical release
45
46 ;Hardware Initialization Constants
47 .equ ROW_DIR_INIT    = $1F          ; Lower 5 pins output (0001 1111) for 5 rows
```



```
48 .equ COL_DIR_INIT    = $00        ; All 8 pins input
49 .equ COL_PULLUP_EN   = $FF        ; Enable internal pull-ups on all columns
50
51 ;Timer 0 Constants
52 .equ T0_COMP_MATCH    = $7C        ; 124 count for 1ms interrupt (with prescaler 64)
53
54 ;Matrix Scanning Constants
55 .equ ROW_0_ACTIVE     = $FE        ; Row 0 active LOW (1111 1110)
56 .equ INITIAL_BASEID   = 0          ; Starting ID for the top-left sensor
57 .equ NO_KEY_PRESSED   = $FF        ; 0xFF indicates an empty scan/no active key
58 .equ COLS_PER_ROW     = 8          ; Number of columns per row
59 .equ TOTAL_SENSORS    = 40         ; Total matrix size (5 rows * 8 cols)
60 .equ FLAG_CLEARED     = 0          ; Used to clear flags and counters
61
```

debouncer/Button Debouncing/init.asm

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;                                                                 ;
3  ;                               init.asm                            ;
4  ;                               Hardware Initialization Setup        ;
5  ;                                                                 ;
6  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
7
8
9  ; Description:           This program tests the sensor functions forHomework #2.
10 ;                      It sets up the stack and calls the homework test function.
11 ;
12 ; Input:                User presses of the switches (sensors) are stored in
13 ;                      memory.
14 ; Output:               None.
15 ;
16 ; User Interface:       No real user interface. The user inputs switch presses
17 ;                      and that data along with the status function output is
18 ;                      written to memory.
19 ; Error Handling:       None.
20 ;
21 ; Algorithms:           None.
22 ; Data Structures:      None.
23 ;
24 ; Known Bugs:           None.
25 ; Limitations:          Only the last (at least) 128 switch inputs are stored.
26 ;
27 ; Revision History:
28 ;   4/30/26  Aaditya Bhat      added init functions
29 ;   5/02/26  Aaditya Bhat      added commenting
30
31
32
33 .cseg
34
35 ; InitPorts
36 ;
37 ; Description:           Configures the I/O ports for the 5x8 sensor matrix.
38 ;
39 ; Operation:             Sets the row pins as outputs and the column pins as inputs.
40 ;                      Enables internal pull-up resistors on the column pins.
41 ;
42 ; Arguments:             None.
43 ; Return Value:          None.
44 ;
45 ; Local Variables:       R16
46 ; Shared Variables:      None.
47 ; Global Variables:      ROW_PORT, ROW_DDR, COL_PORT, COL_DDR
48 ;
```

```

49 ; Input:          None.
50 ; Output:         None.
51 ;
52 ; Error Handling:  None.
53 ;
54 ; Algorithms:      None.
55 ; Data Structures: None.
56 ;
57 ; Registers Changed: R16
58 ; Stack Depth:    0 bytes
59 ;
60 ; Author:          Aaditya Bhat
61 ; Last Modified:   May 2, 2026
62 InitPorts:
63     ;Set the lower pins of ROW_PORT to outputs (0001 1111)
64     LDI    R16, ROW_DIR_INIT
65     STS    ROW_DDR, R16
66     ;Set all pins of COL_PORT to inputs
67     LDI    R16, COL_DIR_INIT
68     OUT    COL_DDR, R16
69     ;Drive the internal pull-up resistors on the column inputs
70     LDI    R16, COL_PULLUP_EN
71     OUT    COL_PORT, R16
72
73     RET
74
75
76 ; InitTimer0
77 ;
78 ; Description:      Configures Timer 0 to generate a compare match interrupt every
79                     1ms.
80 ;
81 ; Operation:        Sets Timer 0 to CTC mode, loads the compare match value, enables
82                     the interrupt, and starts the timer with a prescaler of 64.
83 ;
84 ; Arguments:        None.
85 ; Return Value:     None.
86 ;
87 ; Local Variables:  R16
88 ; Shared Variables: None.
89 ; Global Variables: TIMSK, TCCR0, OCR0
90 ;
91 ; Input:            None.
92 ; Output:           None.
93 ;
94 ; Error Handling:    None.
95 ;
96 ; Algorithms:        None.
97 ; Data Structures:  None.

```

```

98 ; Registers Changed: R16
99 ; Stack Depth:      0 bytes
100 ;
101 ; Author:            Aaditya Bhat
102 ; Last Modified:     May 2, 2026
103 InitTimer0:
104     LDI    R16, (1 << WGM01)
105     OUT    TCCR0, R16                ;Set the timer to CTC Mode
106
107     LDI    R16, T0_COMP_MATCH
108     OUT    OCR0, R16                ;Set the compare match value (124 ct)
109
110     LDI    R16, (1 << OCIE0)
111     OUT    TIMSK, R16              ;Enable the Timer0 Match Interrupt
112
113     LDI    R16, (1 << WGM01) | (1 << CS02)
114     OUT    TCCR0, R16              ;Start the timer (Prescalar = 64)
115     SEI                                ;Ensure interrupt enable
116
117     RET
118
119
120 ; InitVariables
121 ;
122 ; Description:        Clears all shared SRAM variables used by the sensor routines.
123 ;
124 ; Operation:          Sets `SensorFlag`, `SensorState`, `DebounceCnt`, and
125 ;                     `SensorCode` to 0.
126 ;
127 ; Arguments:          None.
128 ; Return Value:       None.
129 ;
130 ; Local Variables:    R16
131 ; Shared Variables:    SensorFlag, SensorState, DebounceCnt, SensorCode
132 ; Global Variables:    None.
133 ;
134 ; Input:              None.
135 ; Output:              None.
136 ;
137 ; Error Handling:      None.
138 ;
139 ; Algorithms:          None.
140 ; Data Structures:     None.
141 ;
142 ; Registers Changed:  R16
143 ; Stack Depth:        0 bytes
144 ;
145 ; Author:             Aaditya Bhat
146 ; Last Modified:      May 2, 2026
147 InitVariables:

```

```
147      CLR      R16      ;Rd <- 0
148                                     ;Init vals with cleared register
149      STS      SensorFlag, R16
150      STS      SensorState, R16
151      STS      DebounceCnt, R16
152      STS      SensorCode, R16
153      RET
154
```

debouncer/Button Debouncing/sensors.asm

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;                                                                 ;
3  ;                      sensors.asm                                ;
4  ;                      Matrix Scanning & Debouncing              ;
5  ;                                                                 ;
6  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
7
8  ; Description:          This file implements the matrix scanning and debouncing routines
9  ;                      for the 5x8 sensor matrix. It includes the Timer 0 ISR, which
10 ;                      scans the matrix, detects key presses, and debounces the input.
11 ;
12 ; Input:                Raw key presses from the 5x8 sensor matrix.
13 ; Output:               Debounced key codes stored in shared memory.
14 ;
15 ; User Interface:      None.
16 ;
17 ; Error Handling:       None.
18 ;
19 ; Algorithms:           Matrix scanning, state machine for debouncing.
20 ; Data Structures:      None.
21 ;
22 ; Known Bugs:           None.
23 ; Limitations:          Supports only a 5x8 sensor matrix.
24 ;
25 ; Revision History:
26 ;   4/30/26   Aaditya Bhat                added Timer0_ISR, state machine, and
27 ;           debouncing logic
28 ;   5/02/26   Aaditya Bhat                added commenting/documentation
29
30 .dseg
31 ; --- Shared Variables in SRAM ---
32 SensorFlag:    .BYTE 1          ; Set to 1 when a valid, debounced key is ready
33 SensorCode:    .BYTE 1          ; Holds the ID (0-39) of the debounced key
34 SensorState:   .BYTE 1          ; Tracks the current phase of the state machine
35 DebounceCnt:   .BYTE 1          ; Counts how many ms the key has been held
36 CurrentRaw:    .BYTE 1          ; The raw key ID currently being debounced
37
38
39 .cseg
40
41
42 ; HaveSensor
43 ;
44 ; Description:          Checks if a valid, debounced sensor activation is available.
45 ;
46 ; Operation:            Reads the `SensorFlag` variable to determine if a sensor is
47 ;                      ready.
```

```

47 ;           If `SensorFlag` is non-zero, a sensor is available.
48 ;
49 ; Arguments:      None.
50 ; Return Value:   Z flag is reset (Z=0) if a sensor is available.
51 ;               Z flag is set (Z=1) if no sensor is available.
52 ;
53 ; Local Variables: None.
54 ; Shared Variables: SensorFlag
55 ; Global Variables: None.
56 ;
57 ; Input:          None.
58 ; Output:         None.
59 ;
60 ; Error Handling:  None.
61 ;
62 ; Algorithms:      None.
63 ; Data Structures: None.
64 ;
65 ; Registers Changed: R16
66 ; Stack Depth:    0 bytes
67 ;
68 ; Author:         Aaditya Bhat
69 ; Last Modified:  May 2, 2026           Z flag is set (Z=1/Equal) if no sensor is
available.
70
71 HaveSensor:
72     LDS     R16, SensorFlag      ;get the sensor flag
73     TST     R16                 ;set Z flag corresponding to sensor flag
74     RET
75
76
77
78
79 ; GetSensor
80 ;
81 ; Description:     Retrieves the debounced sensor code. Blocks until a sensor is
ready.
82 ;
83 ; Operation:       Waits for `SensorFlag` to indicate a valid sensor activation.
84 ;               Reads the `SensorCode` and clears the `SensorFlag` after
reading.
85 ;
86 ; Arguments:       None.
87 ; Return Value:    R16 contains the debounced sensor code (0 to 39).
88 ;
89 ; Local Variables: None.
90 ; Shared Variables: SensorFlag, SensorCode
91 ; Global Variables: None.
92 ;
93 ; Input:           None.
94 ; Output:          None.

```

```

95 ;
96 ; Error Handling:      None.
97 ;
98 ; Algorithms:          None.
99 ; Data Structures:     None.
100 ;
101 ; Registers Changed:   R16, R17
102 ; Stack Depth:        0 bytes
103 ;
104 ; Author:              Aaditya Bhat
105 ; Last Modified:       May 2, 2026
106 GetSensor:
107 GS_Wait:
108     RCALL    HaveSensor
109     BREQ     GS_Wait             ;Loop if Z=0 implies No Sensor Flag
110
111     ;If no branch, sensor is ready
112     LDS      R16, SensorCode     ;Get SensorCode
113     LDI      R17, FLAG_CLEARED
114     STS      SensorFlag, R17     ;clear the flag, acknowledge reading
115     RET
116
117
118
119
120 ; Timer0_ISR
121 ;
122 ; Description:          Executes every 1ms to scan the sensor matrix and handle
debouncing.
123 ;
124 ; Operation:            Scans the 5x8 sensor matrix row by row, detects key presses, and
runs a state machine to debounce the input. Updates shared
125 ;                        variables
126 ;                        to indicate valid sensor activations.
127 ;
128 ; Arguments:            None.
129 ; Return Value:         None.
130 ;
131 ; Local Variables:      R16–R20, CurrentRaw
132 ; Shared Variables:     SensorFlag, SensorCode, SensorState, DebounceCnt
133 ; Global Variables:     None.
134 ;
135 ; Input:                None.
136 ; Output:               None.
137 ;
138 ; Error Handling:       None.
139 ;
140 ; Algorithms:           Matrix scanning, state machine for debouncing.
141 ; Data Structures:      None.
142 ;
143 ; Registers Changed:    R16–R20, flags

```



```

144 ; Stack Depth:      At least 5 bytes
145 ;
146 ; Author:            Aaditya Bhat
147 ; Last Modified:     May 2, 2026
148
149 Timer0_ISR:
150     ;Save Context by Pushing Changed Regs to Stack
151     PUSH    R16
152     IN      R16, SREG
153     PUSH    R16
154     PUSH    R17
155     PUSH    R18
156     PUSH    R19
157     PUSH    R20
158
159     ;Setup Matrix Scanning
160     LDI     R16, ROW_0_ACTIVE      ;Row 0 Active LOW (1111 1110)
161     LDI     R17, INITIAL_BASEID   ;Tracking BaseID for cur row
162     LDI     R18, NO_KEY_PRESSED   ;R18 defaults to 0xFF (No key pressed)
163
164
165
166
167 ; ScanLoop
168 ;
169 ; Description:        Iterates through each row of the 5x8 sensor matrix to detect key
presses.
170 ;
171 ; Operation:          Drives one row low at a time, reads the column states, and
checks for key presses.
172 ;                    If a key press is detected, it calculates the key's ID and exits
the loop.
173 ;
174 ; Arguments:          None.
175 ; Return Value:       None.
176 ;
177 ; Local Variables:    R16 (row selector), R17 (base row ID), R18 (key ID), R19 (column
states), R20 (column counter)
178 ; Shared Variables:   None.
179 ; Global Variables:   ROW_PORT, COL_PIN
180 ;
181 ; Input:              None.
182 ; Output:             None.
183 ;
184 ; Error Handling:      None.
185 ;
186 ; Algorithms:         Iterative row scanning.
187 ; Data Structures:    None.
188 ;
189 ; Registers Changed:  R16–R20
190 ; Stack Depth:       0 bytes

```

```

191 ;
192 ; Author:          Aaditya Bhat
193 ; Last Modified:   May 2, 2026
194
195 ScanLoop:
196     STS     ROW_PORT, R16    ; Drive selected ROW Low
197     NOP                      ; Weird timing-related issue on board, NOP for safety
198     NOP
199
200     IN      R19, COL_PIN     ; Read 8 columns
201     COM     R19              ; Invert bits => pressed keys = 1
202
203     TST     R19
204     BREQ    NextRow         ; If R19 was empty, just go to next row (no presses)
205
206     LDI     R20, 0           ; R20 is col counter
207
208
209
210
211 ; FindCol
212 ;
213 ; Description:      Identifies the column of the pressed key in the current row.
214 ;
215 ; Operation:        Iterates through the bits of the column state register to find
the first active column.
216 ;                  The column counter is incremented until the active column is
found.
217 ;
218 ; Arguments:        None.
219 ; Return Value:     None.
220 ;
221 ; Local Variables:  R19 (column states), R20 (column counter)
222 ; Shared Variables: None.
223 ; Global Variables: None.
224 ;
225 ; Input:            None.
226 ; Output:           None.
227 ;
228 ; Error Handling:   None.
229 ;
230 ; Algorithms:       Bitwise iteration.
231 ; Data Structures:  None.
232 ;
233 ; Registers Changed: R19, R20
234 ; Stack Depth:      0 bytes
235 ;
236 ; Author:          Aaditya Bhat
237 ; Last Modified:   May 2, 2026
238
239 FindCol:

```

```

240     SBRC    R19, 0           ; Skip if BIT 0 is clear
241     RJMP    FoundPress      ; Executes if BIT not clear
242     INC     R20              ; Col+=1
243     LSR     R19              ; Shift to check next bit
244     RJMP    FindCol         ; Recall function
245
246
247
248
249 ; FoundPress
250 ;
251 ; Description:      Calculates the unique key ID based on the current row and
column.
252 ;
253 ; Operation:        Adds the base row ID to the column counter to compute the key
ID.
254 ;                  Exits the scanning loop after identifying the key.
255 ;
256 ; Arguments:        None.
257 ; Return Value:     None.
258 ;
259 ; Local Variables:  R17 (base row ID), R18 (key ID), R20 (column counter)
260 ; Shared Variables: None.
261 ; Global Variables: None.
262 ;
263 ; Input:            None.
264 ; Output:           None.
265 ;
266 ; Error Handling:   None.
267 ;
268 ; Algorithms:        Arithmetic calculation.
269 ; Data Structures:   None.
270 ;
271 ; Registers Changed: R18
272 ; Stack Depth:      0 bytes
273 ;
274 ; Author:           Aaditya Bhat
275 ; Last Modified:    May 2, 2026
276
277 FoundPress:
278     ; Calculate the exact button ID (0 to 39)
279     MOV     R18, R17
280     ADD     R18, R20
281     RJMP    ScanDone        ; Stop scanning. Handle one key at a time.
282
283
284
285
286 ; NextRow
287 ;
288 ; Description:      Advances to the next row in the sensor matrix for scanning.

```

```

289 ;
290 ; Operation:          Updates the row selector and base row ID to prepare for scanning
the next row.
291 ;                   Checks if all rows have been scanned and loops back if
necessary.
292 ;
293 ; Arguments:          None.
294 ; Return Value:      None.
295 ;
296 ; Local Variables:    R16 (row selector), R17 (base row ID)
297 ; Shared Variables:  None.
298 ; Global Variables:  None.
299 ;
300 ; Input:              None.
301 ; Output:             None.
302 ;
303 ; Error Handling:     None.
304 ;
305 ; Algorithms:         Iterative row advancement.
306 ; Data Structures:   None.
307 ;
308 ; Registers Changed:  R16, R17
309 ; Stack Depth:       0 bytes
310 ;
311 ; Author:             Aaditya Bhat
312 ; Last Modified:     May 2, 2026
313
314 NextRow:
315     ;Find ButtonID
316     SUBI    R17,-COLS_PER_ROW    ;Add 8 to ROW ID
317     SEC                                ;set carry flag
318     ROL     R16                    ;since carry set, 1101 => 1011 => 0111 etc
319     CPI     R17, TOTAL_SENSORS    ;checked all 5 rows?
320     BRNE    ScanLoop              ;jump if not done, continue if done
321
322
323
324
325 ; ScanDone
326 ;
327 ; Description:        Handles the transition from matrix scanning to the debouncing
state machine.
328 ;
329 ; Operation:          Reads the current state of the debouncing state machine and
branches to the appropriate handler.
330 ;
331 ; Arguments:          None.
332 ; Return Value:      None.
333 ;
334 ; Local Variables:    R16 (state machine state)
335 ; Shared Variables:   SensorState

```

```

336 ; Global Variables:  None.
337 ;
338 ; Input:             None.
339 ; Output:            None.
340 ;
341 ; Error Handling:     None.
342 ;
343 ; Algorithms:         State machine branching.
344 ; Data Structures:   None.
345 ;
346 ; Registers Changed: R16
347 ; Stack Depth:       0 bytes
348 ;
349 ; Author:             Aaditya Bhat
350 ; Last Modified:     May 2, 2026
351
352 ScanDone:
353     ;Debouncing StateMachine
354     LDS     R16, SensorState
355     CPI     R16, STATE_IDLE
356     BREQ    Handle_Idle                ;jump to idle state execution if in idle
state
357     CPI     R16, STATE_DEBOUNCE
358     BREQ    Handle_Debounce            ;jump to debounce state execution if in
debounce state
359     CPI     R16, STATE_WAIT_RELEASE
360     BREQ    Handle_WaitRelease         ;jump to wait release state if waiting for
release
361     RJMP    ISR_End                    ;otherwise end interrupt service routine
362
363
364
365
366 ; Handle_Idle
367 ;
368 ; Description:        Handles the idle state of the debouncing state machine.
369 ;
370 ; Operation:          Checks if a key press is detected. If so, transitions to the
debounce state and initializes variables.
371 ;
372 ; Arguments:          None.
373 ; Return Value:       None.
374 ;
375 ; Local Variables:    R16 (state machine state), R18 (key ID)
376 ; Shared Variables:   SensorState, CurrentRaw, DebounceCnt
377 ; Global Variables:   None.
378 ;
379 ; Input:              None.
380 ; Output:             None.
381 ;
382 ; Error Handling:     None.

```

```

383 ;
384 ; Algorithms:      State transition.
385 ; Data Structures: None.
386 ;
387 ; Registers Changed: R16
388 ; Stack Depth:    0 bytes
389 ;
390 ; Author:          Aaditya Bhat
391 ; Last Modified:   May 2, 2026
392
393 Handle_Idle:
394     CPI        R18, NO_KEY_PRESSED
395     BREQ       ISR_End ;Nothing pressed
396
397     STS        CurrentRaw, R18           ;deposit key id
398     LDI        R16, STATE_DEBOUNCE
399     STS        SensorState, R16         ;prepare debounce state
400     LDI        R16, FLAG_CLEARED
401     STS        DebounceCnt, R16         ;prep counter
402     RJMP       ISR_End                  ;end current interrupt SR
403
404
405
406
407 ; Handle_Debounce
408 ;
409 ; Description:      Handles the debounce state of the debouncing state machine.
410 ;
411 ; Operation:        Verifies if the key press is stable for the required debounce
time.
412 ;                  If stable, transitions to the wait-release state and updates
shared variables.
413 ;
414 ; Arguments:        None.
415 ; Return Value:     None.
416 ;
417 ; Local Variables:  R16 (debounce counter, state machine state), R18 (key ID)
418 ; Shared Variables: SensorState, DebounceCnt, SensorCode, SensorFlag
419 ; Global Variables: None.
420 ;
421 ; Input:            None.
422 ; Output:           None.
423 ;
424 ; Error Handling:   None.
425 ;
426 ; Algorithms:       State transition, time-based validation.
427 ; Data Structures:  None.
428 ;
429 ; Registers Changed: R16
430 ; Stack Depth:      0 bytes
431 ;

```

```

432 ; Author:          Aaditya Bhat
433 ; Last Modified:    May 2, 2026
434
435 Handle_Debounce:
436     CPI        R18, NO_KEY_PRESSED
437     BREQ       Reset_To_Idle          ;key pressed early
438
439     LDS        R16, CurrentRaw
440     CP         R18,R16
441     BRNE       Reset_To_Idle          ;switched keys early
442
443     ;Valid Continuous Press (neither above cases happened)
444     LDS        R16, DebounceCnt
445     INC        R16                    ;since valid press, increment the count
446     STS        DebounceCnt,R16       ;deposit updated count to memory
447     CPI        R16, DEBOUNCE_TIME    ;determine if our debouncecount has reached
threshold
448     BRNE       ISR_End                ;not Fully debounced, just end
449
450     ;Debounced Time Reached
451     LDI        R16, STATE_WAIT_RELEASE ;prepare sensor for release
452     STS        SensorState, R16       ;save state
453
454     LDS        R16, CurrentRaw
455     STS        SensorCode, R16        ;update valid sensor code
456
457     LDI        R16, $01
458     STS        SensorFlag, R16        ;update sensor flag (button has successfully
debounced)
459     RJMP       ISR_End                ;end routine
460
461
462
463
464 ; Reset_To_Idle
465 ;
466 ; Description:      Resets the state machine to the idle state.
467 ;
468 ; Operation:        Sets the state machine to idle and clears any intermediate
variables.
469 ;
470 ; Arguments:        None.
471 ; Return Value:     None.
472 ;
473 ; Local Variables:  R16 (state machine state)
474 ; Shared Variables: SensorState
475 ; Global Variables: None.
476 ;
477 ; Input:            None.
478 ; Output:           None.
479 ;

```

```

480 ; Error Handling:      None.
481 ;
482 ; Algorithms:          State reset.
483 ; Data Structures:     None.
484 ;
485 ; Registers Changed: R16
486 ; Stack Depth:        0 bytes
487 ;
488 ; Author:              Aaditya Bhat
489 ; Last Modified:       May 2, 2026
490
491 Reset_To_Idle:
492     LDI        R16, STATE_IDLE    ;prepare idle state
493     STS        SensorState, R16   ;set idle state
494     RJMP       ISR_End            ;end interrupt routine
495
496
497
498
499 ; Handle_WaitRelease
500 ;
501 ; Description:          Handles the wait-release state of the debouncing state machine.
502 ;
503 ; Operation:            Waits for the user to release the key. Once released,
504 ;                       transitions back to the idle state.
505 ;
506 ; Arguments:            None.
507 ; Return Value:         None.
508 ;
509 ; Local Variables:      R16 (state machine state), R18 (key ID)
510 ; Shared Variables:     SensorState
511 ; Global Variables:     None.
512 ;
513 ; Input:                None.
514 ; Output:               None.
515 ;
516 ; Error Handling:       None.
517 ;
518 ; Algorithms:           State transition.
519 ; Data Structures:      None.
520 ;
521 ; Registers Changed: R16
522 ; Stack Depth:        0 bytes
523 ;
524 ; Author:              Aaditya Bhat
525 ; Last Modified:       May 2, 2026
526
527 Handle_WaitRelease:
528     CPI        R18, NO_KEY_PRESSED ;is there still user input from any sensor?
529     BRNE       ISR_End            ;branch if user still holding key

```



```

529
530     ;User Let Go – back to idle
531     LDI     R16, STATE_IDLE     ;prepare idle state
532     STS     SensorState, R16   ;set idle state
533     RJMP    ISR_END            ;end routine
534
535
536
537
538 ; ISR_END
539 ;
540 ; Description:      Restores the processor context and exits the interrupt service
routine.
541 ;
542 ; Operation:       Pops all saved registers from the stack and returns from the
interrupt.
543 ;
544 ; Arguments:       None.
545 ; Return Value:    None.
546 ;
547 ; Local Variables: None.
548 ; Shared Variables: None.
549 ; Global Variables: None.
550 ;
551 ; Input:           None.
552 ; Output:          None.
553 ;
554 ; Error Handling:   None.
555 ;
556 ; Algorithms:      None.
557 ; Data Structures: None.
558 ;
559 ; Registers Changed: None.
560 ; Stack Depth:     Restores all saved registers.
561 ;
562 ; Author:          Aaditya Bhat
563 ; Last Modified:    May 2, 2026
564
565 ISR_END:
566     ;Restore Context from earlier deposit, reverse order (stack LIFO)
567     POP     R20
568     POP     R19
569     POP     R18
570     POP     R17
571     POP     R16
572     OUT     SREG, R16
573     POP     R16
574     RETI
575

```